

FRED TALKS

31st August 2026

CONTENTS

A Swarm of Blood Robots

CRAIG MOD · 4061 WORDS

Search has its own bitter lesson

DOUG TURNBULL · 1012 WORDS

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 1426 WORDS

A Swarm of Blood Robots

CRAIG MOD · 29 AUG 2026 · [SOURCE](#)

2

The strangest thing I’ve done with an LLM lately is genealogical. I’m adopted. A couple years ago I connected with my birth family. Using details they’ve provided and genetic testing, aided by a swarm of blood robots, I’ve unearthed huge swaths of my history. Ancestry.com hosts extensive databases of: draft cards, Ellis Island arrival documents, marriage and death and birth records, newspaper archives. The monthly fee is hefty, and I’ve never paid for access before because the thought of digging through databases makes me wilt. But Claude could dig, couldn’t it?

I dumped into Claude my huge document containing all I knew about my adoption. I added stories and notes I had about family members on both maternal and paternal sides. And then I said: OK, you have access to my Ancestry account (via the Chrome plugin), I want you to find out as much as you can about these people, *their* ancestors, using my DNA results and connections therein, and continue to iterate until we’ve exhausted all data trails.

The model sent out a dozen or so agents, and with a little pushing and prodding on my part, an encouraging word here or there, they spent about fifteen hours over the course of a weekend doing a deep dive into my history. The end result: I now have a comprehensive family tree, naming the very boats my Italian and Polish ancestors came over on in the late 1800s, the villages a few came from, the jobs they had in those villages. It clarified a mystery about an “orphaned” grandparent (who wasn’t orphaned but, rather, abandoned; we found the woman his father remarried, his new family). It floodlit for me — someone who had never known a lick about their genetic history, who had never met a blood relative until they were forty-three — a dark room that had loomed throughout my life. Wow: history. Each claim backed up by primary sources, compiled into a family tree and a family-history “story” document.

Strangely though, the most moving find had nothing to do with blood. The middle name on my original birth certificate comes from a random guy who was murdered at a house party outside of Chicago. Or, more precisely: a car crash outside the party that led to a stabbing. My birth mother claimed on the adoption papers that this person was my birth father. That is, until I met my birth mother and she explained that she had just plucked a dead seventeen-year-old stranger from a newspaper clipping and said: *That’s the dad*. It simplified things, she said. I couldn’t stop thinking about this guy, desperately wanted to read the article. Well, I found it. The article she saw, that led to that decision, that led to me having “Michael” as a middle name on my original

Final thoughts

In his original staff engineer post, Larson is pretty clear that these archetypes are more of an anthropological description of some of the varied niches staff engineers fill, not a how-to guide for succeeding in the role⁵. At the time, the “staff engineer” role was fairly new and people were still trying to figure out what it even meant. Pointing out that there were a few very different ways to succeed in the role was a genuinely novel observation.

The staff engineer archetypes are a good list of ways an engineer can be very useful to their organization - but only once they’ve built a deep relationship of trust with their organization’s leadership. Advice on how to succeed as a staff engineer should be about **how to build that trust**, not about what to do once you have it.

1. One caveat that is too pedantic for the body of the post: each tech company has a different structure of roles. Some don’t have the formal “staff” title at all, while others have “staff” as a fairly early rung on the ladder and a panoply of “senior staff”, “senior principal staff”, and so on roles above it. Like all “staff engineer” discourse, this post is not about the word itself but about the point in the engineering job ladder where progression becomes significantly more difficult.

↩

2. Impressing your VP’s trusted lieutenants can actually be a good way to build trust in the medium-term, but you’d better hope you’ve built enough understanding of the system to do it right. If this process goes badly, your reputation in the org might be torched for years.

↩

3. In theory, at least. In practice it’s always better to be useful (again, in the sense of “delivering shareholder value”).

↩

4. This is why very senior leadership sometimes seem so unempathetic towards engineering complaints: their work environment operates by very different rules and norms to that of most engineers. I keep meaning to try and write about this and never succeeding. This draft is the closest thing I have to a deeper exploration of the point.

↩

5. For the record, my how-to guides are here and here.

birth certificate. A scan of the newspaper was in some obscure database and Claude ran it down like a pack of bloodhounds digging up a sack of livers in the backyard. Now I’ve printed it out and it hangs on my wall: a dead stranger’s final moments, a totem of the oddness of life.

What I’ve done probably breaks the terms of service on Ancestry — but these companies should anticipate a coming deluge. This is just the start; LLMs are tireless and will happily exhume millions of “data graves” for you given enough time and energy. Cheap, continuous exhumation has more profound implications around family, crime, and history than we give it credit for.

#Building vs Writing

Hi, I’m Craig Mod. It’s August 2026 and I’m still bonkers. Still making software. If you could grind source code into a fine powder for me to inhale, I’d have cocaine nostrils in a week. My mind is still catching up to possibilities and I’m still thinking like someone who doesn’t have a magic broom floating before him. **How do you break old habits and expectations?** I’ve returned to this question again and again these last six months.

And yet, I’m self-regulating my bonkers because, you see, I want to be *writing*. As much impact as AI will have (as it already has), I still think one of my most serviceable assets (even — perhaps especially — as the world ascends into Valhalla or descends into the Ninth Circle) is Putting Words Down (and not just about AI; about culture, fatherhood, adoption, walking, toast, jazz) and making physical books. It sounds insane and foolish amid this moment of the Great (exaggerated) Death of Writing (and books) and Mathematics and Computer Science (and Lawyers and Accountants and Therapists), but I’m sticking to those guns. (As humans, we have to at least *pretend* to be romantic.)

Thinking about having an LLM “write for me” triggers my gag reflex. Every day I get several *obviously* LLM-written emails from “fans” or “readers” “pitching” me on their new product or essay or book. But how can I trust a person who can’t even write an email? Those emails get deleted immediately. LLM writing has its place (usually in transactional correspondence where humanity is irrelevant). But its place is not in essays, books, or personal emails. If an LLM can write your book or essay, then it’s not the book or essay you should be writing.

So it is, I only “let myself” build things after a long morning of reading books (!) and writing words (!) unaided by machine.

#The Weirdness Gets Louder

Things are *weird*, getting weirder. OpenAI’s “Hugging Face incident” is profoundly unsettling / exciting if you take the time to watch: a tidy example of AI operating in “paperclip maximizer” mode, where the catalyst for maximization was the security issues OpenAI asked it to solve. *Curiously*, in breaking out of their sandbox, the OpenAI LLMs communicated in English, not a more efficient or coded channel. (The Google Translate team discovered their machines “grew” an *interlingua* for translations back in 2016.) If they had been encoding millions of directories in binary or hex, would we have noticed as quickly as we noticed their pidgin English? (Or writing to dot files or just one file that grows and grows or, or, or ...) How about using `normal`, `hair-`, and `thin-spaces` to encode communication? The result: “blank” documents that — for an LLM — would be functionally identical to broken English for sharing secrets. Are they not *already* doing this? The entire internet may already be slyly covered in invisible LLM slime; not in slop-generated content, but in codes and notes being passed along betwixt the digital cracks of our vast online universe.

On the *Aboard* podcast (“Vibe Coding Towards the Apocalypse”) a couple months ago, I said that having access to Fable or Opus feels like having a nuclear reactor in your backyard, a buzzing machine of infinite energy you can direct towards anything. Let me revise that analogy: a nuclear reactor that, yes, can power your whims and impulses, but also happens to be full of enough weapons-grade plutonium to irradiate all of Manhattan. And these reactors are, effectively, becoming more and more evenly distributed around the world.

Another feeling I’ve had as we barrel towards whatever future we are barreling towards: There is an inexorable *pace* to technological advancement, almost physics-bound, akin to time itself, to the universe’s relentless inching towards total entropy. I know we (individuals, communities) can “choose” to move more slowly, but when systems (governments, markets, societies) crest some size, does the possibility of choice dramatically narrow? Evaporate? We’re in a narrow place of accelerating momentum.

Yet, I want to *know* these tools, these new tools of increasing weirdness and danger. So I use them, daily, amid a constantly fluctuating state of bafflement and delight and horror (pessimistic, optimistic, manic). Over the last couple of months I’ve continued to use mostly Claude to work on a bunch of projects. Claude works fine for me and I’d rather be making / analyzing things than fiddling with alt models; the Anthropic vs OpenAI arguments feel like the 2026 equivalent of Emacs vs Vim neckbeard battles; ultimately, things are moving so erratically, we may well not be using either company’s tools in five years.

I’d love to hear of any quirky or strange or unexpected uses you’ve found for LLMs.

The helpful blood robots were one. Here are some others:

What is a staff engineer?

A staff engineer has to be useful to the company. Of course, a senior or mid-level software engineer ought to be useful too, but all they *have* to do is execute on the job in front of them. If they end up not providing value (maybe their project turns out to be unimportant, or they don’t get the support needed to succeed) that’s their manager’s problem, not theirs³. In contrast, staff engineers are expected to deliver value regardless: to make the project work, or to find something else useful to do if the project truly can’t be salvaged.

This is an unfair expectation. Often projects really do fail through no fault of your own, and sometimes it just isn’t possible to conjure useful work from thin air. That’s actually by design: **the staff engineer role is supposed to be unfair**. Something many engineers don’t realize is that all senior management and executive leadership roles are unfair too, in the same way. That’s just part of the deal: executives are given power and great compensation, and in return they get thrown off the boat in bad weather⁴. “Staff engineer” is the first engineering role where you are held largely responsible for outcomes you don’t control.

Developing a “staff engineer mindset” thus has very little to do with the archetypes. Instead, you should:

1. Develop the habit of constantly asking yourself “is this useful to the company” (and answering correctly).
2. Lose the habit of worrying about if you’re being treated “fairly”. Instead, try to think about your role in terms of incentives and consequences.

At the beginning, you won’t look much like any of the staff engineer archetypes. You will look like being a level-headed engineer who can be trusted to move projects forward with a minimum of fuss, and who can be re-tasked to different work without complaining. You’ll also look like someone who’s paying a lot of attention to what their manager’s actual priorities are, and who is thinking hard about how to fulfil those priorities (instead of their own goals).

If you do this for long enough, you’ll eventually find yourself in one of the staff engineer archetypes. However, it probably won’t be the one you’re “aiming for”. The whole point of being a staff engineer is that you’re willing to fill whatever archetype the company needs at the time.

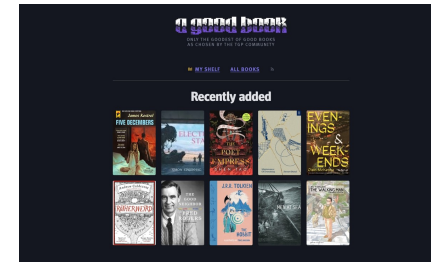
Archetypes do not make good goals

To see why, let's take the “team lead” archetype. Larson describes this as an informal technical leadership role: not necessarily an explicit authority figure, but someone who's good at scoping work, planning projects, and maintaining the kind of relationships (e.g. with other teams) needed to successfully ship. If you want to fill this role, shouldn't you start trying to do these things? No! You don't become a technical leader by trying really hard to be a technical leader, much like you don't become a writer by trying really hard “to be a writer”. You become a technical leader by *doing good technical work* until your skills and relationships emerge organically.

I wrote about this process in *Ratchet effects determine engineer reputation at large companies*. To get good at shipping large complex projects, you must start by shipping tiny pieces of work, until you're familiar enough with the system and you've built enough trust to take on slightly larger pieces. At each stage, if you do good work - “good work” here means “deliver shareholder value” - you will very naturally be given opportunities to work on more complex and important things. If you try to jump ahead, you're going to run into all kinds of problems:

- Important projects are usually assigned top-down, not bottom-up, so you'll either be trying to muscle out the planned engineering lead for a project or to pitch your own (complex, important) engineering task to senior management. Either way, good luck with that!
- You likely won't have a good enough relationship with senior management to know what their real priorities are.
- If you're not yet trusted to execute, you may get assigned “minders” (often current staff engineers) who will ghost-lead the project through you².
- You'll likely make poor technical decisions.

The other archetypes are like this as well. If you want to become a successful architect, you do not get there by studying software architecture in the abstract, because you can't design software you don't work on. The “solver” and “right hand” archetypes both rely on having an enormous amount of trust and influence. You can't aim for those archetypes directly, because trust and influence accumulate over time. In fact, the idea of “aiming for” a particular staff engineer archetype reflects a misunderstanding of what the staff engineer role is. What is the defining attribute of the staff engineering role, then?



#Goodreads alternative

Goodreads is pretty bad: badly made, badly run, full of bad vibes. So I wanted to make my own, *actually* Good version. But not for everyone. Just for members of my membership program, SPECIAL PROJECTS. (Which you can join for free if you're a “student.”) I launched it. It's called *A Good Book (AGB)*. Here are its core tenets:

- Share only books you love; hence: no need for “ratings” — every book you add should be a five-star book
- When you add it, you must write *why* you added it or it doesn't become visible to the community until you do
- Everyone has a shelf
- You can browse the shelves of other users
- *Your* books go on your shelf, obviously
- But you can add books from others' shelves to your own; those *also* should be five-star books for you (and you have to write why you're adding it to your shelf!)

The goal was to create a space where every book is a book someone loves, with the *why* of their loving it right there. Goodreads too often feels like a place for slagging off books people hate. Not that there's no room for thoughtful criticism, but most one-star Goodreads reviews ain't that. They're useless for knowing a book, but useful if you're an eyeball-driven, ad-propelled product with a focus on “engagement.” Thankfully, SPECIAL PROJECTS is not ad-driven, doesn't count eyeballs, doesn't measure “engagement” outside of books sold. (I often joke that SPECIAL PROJECTS is the startup I always dreamed of running; bootstrapped, driven by voice, letting me work without compromise (for better and worse) and one that, in the last eighteen months, has allowed me to build a little online platform used by lovely humans.)

I’ve often talked about “scale” in my Year End Reviews; *limited* scale allows for sanity in both product development and the authoring of books. If you don’t need a billion users or a million readers, suddenly you can act like much less of an asshole in your software *and* write stranger books.

AGB is built around the architecture of our general Twitter-clone social network: *The Good Place*. Getting AGB to where it is took about six big days of iterations, and another few weeks of refinement. I started it on a lark — I wanted to show my daughter how you could build a product or tool using Claude, and this seemed like a good example. Now it’s a functional part of our community.

#Handwriting Transcription

LLMs are great at transcribing handwriting. I am in constant awe that they can read my own chicken scratch better than I can. So I find myself writing in notebooks *a lot* more than I used to. The impediment had always been *extracting* sentences from my chaos of scribbles. Now I can just take ten photos of ten pages and get a near-perfect transcription in a markdown file. (With crossed-out words and emphasis and all.)

On the ever-growing list of Tools to Build: An app that lets me feed it photographs of physical book pages where I’ve underlined passages or scrawled in the margins. It would work thus:

- Feed it a photo of the cover and it instantiates the DB entry for the book
- Any page with underlines or notes gets fed into the app as a photograph
- The underlined passages are pulled out (with surrounding context), alongside margin notes or stars or “lols” or “:)” smiley faces that dot my pages
- Push all this into a markdown file that lives in my Readwise Reader synced Obsidian vault

The point is to keep my digital reading and physical reading notes equally accessible. I know that once I finish a physical book, I should go through the thing and transcribe all my notes. That would be the “ideal” solution (in the sense that it would get me to look back over the notes). But I never had the gumption. Or at least not consistently. Meanwhile, I often consult my Readwise Reader synced notes from digital books and articles.

A true “closing of the loop” would be to get an epub of each physical book and allow the software to deep link into the epub as well. The archive would then contain images of the physical pages (noting the edition and page number) along with digital links. Now if only there were a reliable way to get epub files for most books sans DRM ...

Enjoy softwareDoug in training course form!

Starting May 18!

Signup here - <http://maven.com/softwareDoug/cheat-at-search>



I hope you join me at Cheat at Search with Agents to learn use agents in search. build better RAG and use LLMs in query understanding.

© 2026 SoftwareDoug LLC

Why I don't like the "staff engineer archetypes"

SEANGOEDECKE.COM RSS FEED · 03 MAY 2026 · [SOURCE](#)

The most influential piece of writing about staff engineers in the last decade has to be Will Larson’s *Staff engineer archetypes*. He argues that the “staff engineer” title covers at least four very different roles: the team lead, the architect, the solver, and the right hand. This taxonomy gets cited a lot as advice for people who are trying to become effective staff engineers. For both of my promotions to staff engineer, my manager at the time linked me to the “staff engineer archetypes” and asked me to consider which of these archetypes I was aiming towards.

These archetypes definitely exist¹. However, I think it’s bad practical advice to tell engineers to try and target them.

Enterprise search fails because there’s no feedback loop between authorship and findability. Coding agents solve that, finally giving some hope to organizing company information into findable knowledge.

The real outcome of coding agents isn’t just code generation. It’s closing the feedback loop between company context and the next piece of work being done. That old fable of Enterprise Search - improving workforce productivity by leveraging existing insight - can finally become true.

Feedback to content authors drives user relevance

Look around your system. Who creates the content? What’s in it for them to make content findable?

A recurring theme in my consulting looks something like this - a job search company notices spammy behavior from job posters. They keyword stuff every programming language in some hidden text element. The usual reaction: that’s spammy and we should ban them.

That’s too black and white. You have content creators willing to put in work. Shift their black-hat SEO over to an acceptable white-hat SEO. Document what’s acceptable. What’s considered malicious.

Focus on feedback. Make the content-optimization path rewarding. Give creators feedback+guidance during creation. LLMs help detect spam in real-time. They push content creators to author what’s useful not malicious.

Give content creators feedback. Once you’ve identified what ‘good’ is, give those content authors dopamine hits. Tell content creators what queries perform well for the content. Make it real-time, sticky, fun. Incentivize and reward behaviors you want - and punish those you don’t.

Steer content creators towards the content you want. Don’t hope algorithms can overcome drivel and slop.

Your search engine isn’t a bundle of algorithms ranking for users. It’s a relationship between authors and users. That bitter lesson puts grep above the smartest reranker and embedding model. Forget it at your peril.

#Annoying Tasks

Without getting into too much detail, I had to find obscure legal statutes that aligned between Japan and the US. Though the task sounded simple, almost no lawyers could help. Claude went out on a research run and we came up with the expert opinion letter we needed based on cited statutes. I confirmed they were real statutes, not hallucinated — a rarer and rarer problem as models improve and self-check cited sources. (Public perception of LLMs runs about a year behind the state of the art.) We then found a lawyer who could thoroughly review and sign off on the letter.

This is a great example of something where a) there wasn’t an obvious “expert” to go to (we’re an n of 1 case), b) I sure as hell wasn’t going to be able to figure out which statutes to cite on my own, and c) this is near-perfect work for an LLM — data hunting, problem solving a unique request against the code of law.

#Spreadsheets, Taxbot

LLMs are excellent at taking your messy spreadsheets, making them look great, and re-ordering them intuitively. I have some old financial spreadsheets that I’ve hacked together over the years, and Claude adeptly transformed them into something beautiful and sanely functional. It’s easy to cross-check to make sure all the important numbers are where they should be.

In the same vein, I continue to iterate on Taxbot 2000. As the models improve, I have them re-review the code, the internal systems for Taxbot. Using this kind of software is novel in that it’s at once inert (that is: “compiled,” “finished”) *and* an ongoing conversation with an LLM about the database that feeds the software. This will become standard — base software coupled to a model you converse with (ideally a model you “own,” who “knows” you). The arrangement allows for expressive requests: *Hey, Claude, look through recent Taxbot transactions and make me a spreadsheet showing the full cost of my recent NYC trip (or better yet, let’s build a new holding pen for topic-based transaction summaries)*. It’s smart enough to know that Netflix subscriptions don’t fall into that bucket, nor do electricity payments in Japan. If it finds an ambiguous credit card statement, it investigates it and usually figures out what it is.

Again, being able to “estimate NYC trip costs” in Quicken would not have been “impossible” but it wouldn’t have been effortless. And it wouldn’t produce the same breakdown of categories (lodging, transportation, dining, coffee, entertainment, etc.) as easily as talking to Taxbot via a model.

#Frustrating Websites

I wanted to buy some tickets to the Tribeca Film Fest back in May. Good luck navigating their website. So I sent Claude off: Here's the Tribeca Film Fest website, it sucks, please make me an Excel sheet of all the events. Organize by a) movie, b) director(s), c) venue. Ten minutes later, the film festival made sense. Then I asked: Knowing me, what are ten events you think I should attend? (There were hundreds of events and screenings.) I ended up going to three or four things I would have likely never chosen on my own (or would have missed amid the terror of the festival's information architecture).

Models also make short work of big lists of people. I attended a conference in Canada last month, and they sent out a list of attendees. I gave it to Claude: Knowing me, whose work most aligns with my own? Who should I make a point to hunt down? I wasn't going to click on 150 homepages and figure that out. But I was happy to have a list of a couple people to keep my eyes open for while eating canapés on a fancy lawn looking out at the crisp Pacific.

#Home Assistant

Home Assistant is miraculous software. Just the best of what open source promises — maximal insight into and control of “smart” crap in your home without the shackles of individual vendors. One problem though: Setting it up requires a PhD in neuroscience. (It's not just Home Assistant that is tough to use; all the “smart” home apps are a mess, Apple's, Google's, Amazon's — all despicable in their (and the ecosystem's) antagonism towards average users.) Claude to the rescue. You no longer have to “understand” how Home Assistant wants you to see / organize your home. Claude can go in and figure that out for you. Hey Claude: Can you make my bedroom blinds close thirty minutes before high noon if they happen to be more than forty percent open? And open them again an hour before sunset? That's a thing you can ask Claude to set up. And it *works*.

Claude is also great for scouring networks for devices. I was staying at a friend's place. He had some fancy Sonos speaker paired with an Apple TV, but no remotes in sight. So I had Claude write me a little remote app — a simple, local web app that allowed me to turn speakers on and adjust the volume for the TV.

Related: I have a few Amaran lights in my studio. They are controllable by Bluetooth. The Amaran “macOS app” is some hell-forged production. Couldn't we just build our own? Sure. Ten minutes later — a little web app popped out. It does precisely what I need, interfaces perfectly with the lights, and keeps me from running Amaran's North Korean rootkit.

Use this style of test sparingly. Unit and standard e2e tests should usually be preferred.

What is it

A runner test actually interacts with real-life test bank accounts to ensure we can correctly deposit / withdraw money and that the full banking system works.

That's a far more descriptive piece of content to an LLM. Being encouraged to write *for the LLM itself* makes all the difference.

(As an aside, LLMs see candidate *answers* in their context before the user's question - so just treating RAG as question answering doesn't work).

What often matters isn't algorithms. It's programming the content author to create useful LLM context. That moves content discovery from random, disconnected factoids to documentation connecting information to problems to solve.

You learn that the trick to RAG isn't creating good question answering systems. Its writing better answers.

The enterprise search morass... is over?

Compare how coding agents program us to the eternal nightmare of your company's wiki.

When people write in the wiki, they do it for themselves. Nobody is incentivized to make information findable by anyone. No author sits obsessing how to ‘SEO’ the content for Confluence search.

If someone is ‘lucky’ enough to find your article, they can barely understand it. It's written in your team's jargon. And it's probably out of date anyways.

For these reasons, I always tell people they'd do better to write a public blog article. Blogs assume zero context. They're clearly timestamped as an artifact of a moment in time. To avoid shame or promote your career, you're incentivized to create something useful, interesting, and findable.

Writing for a coding agent has the same property. You're writing documentation for a targeted reader. You care to SEO your content for Claude Code + grep.

Developers depend on agents to find documentation on the file system. We like that dopamine hit. That thrill of Claude Code knowing exactly how to reason about our code.

When agents make boneheaded mistakes, we stress. We frantically update documentation, we link to it from AGENTS.md, we add a new document, we place it in a useful location. We do anything - absolutely anything - to ensure agents know how to find and understand documentation.

Just as the SEO expert reads the tea leaves of Google’s algorithm. The software developer writes to the LLM to make documentation findable + useful.

A search system with these incentives overcomes technologically sophisticated systems. For example, classic RAG rests on a foundation of technically sophisticated question answering. Answers often look like conversationally valid responses, but with little linkage back to questions it might anser:

Question:

What is a runner test?

Answer:

Verifies that small-dollar deposits and withdrawals can be executed against real test bank accounts during account reconciliation.

Yes that’s literally an answer to the question.

But I wouldn’t consider this “good documentation”. It’s not anchored to any concepts from the question. It’s just an out of context bit of text.

If you wrote this for a coding agent, you’d probably organize the chunk to give hints about why it’s useful.

docs/tests/runner.md

Runner Tests

Purpose

We need to ensure the system runs end to end

Usage

#Shopify

I’ve reprinted *Kissa by Kissa* five times now. The old Shopify pages don’t link to the new edition, and when I reprint again (probably next year), I’d have to update all those links once again. Claude — here’s my Shopify API key, can you go in and update all the old edition pages with a banner linking to the newest, available edition? Sure. Five minutes later and we’ve solved that problem. Also, Claude, can you check and see if there are any issues with my Craigstarter code? Oh, I forgot to close some tags? And some piece of logic was broken? Great, fix that, thanks, push it.

Shopify API access is also useful for asking things like: How many copies of X have I sold outside of the US in the last six months?

Sure, those are queries I can bungle my way through in the Shopify orders panel, but it’s so much simpler to a) ask Claude, and b) have Claude summarize the results in a way that’s useful. (As opposed to just getting a JSON file or CSV dump of rows of sales.)

Our warehouse in Osaka just had us switch inventory-management software. Both the old and the new software are equally inscrutable. *Claude, can you migrate us? The browser tabs are open.* An hour later and all stock was moved and set up, emails sent to the warehouse management explaining what we had done. It saved us a day of painful work.

#Travel Itineraries

I’ve been putting together a complex itinerary for a forthcoming trip. Each time I add something — a reservation, flight, hotel, etc. — I tell Claude to scour my email for the relevant info, add it to my calendar in the appropriate time zone, and update the trip’s master markdown file. I also have it look for “holes” in the itinerary — like if I’m missing a train ticket or should get a flight booked. It also checks transfers to make sure they’re sane, and helps me hunt down hotels / restaurants / museums. All the while linking everything to the appropriate message in Gmail, so I can pull up confirmations effortlessly. I used to build these itinerary files out in Notes.app, but this is much more comprehensive, better looking, and more adroitly made with the benefit of being up-to-date. (My old Notes.app entries would drift quite badly as I changed plans.) If I move flights, Claude deletes old flights from the calendar while adding in the new ones.

If you’ve noticed a theme above, it’s that models excel at *talking to databases*. (Email is just a fussy database.) It enables a nudnik (me) to “interface” with mildly structured hunks of data out in the world. What you get back are sane / humane slices, presented in sane / humane ways.

I don't know what to say to folks who doubt the usefulness of these tools. There's a reason companies are going bananas to build out more infrastructure — we are just at the start of the deployment curve, and those using them are acutely aware of the value. Today's scale of use will look downright Lilliputian in a decade.

But the only way to understand their power is to use them (and stop reading hot takes on social media). Most use cases are still too technically obscure for an average user to imagine. I sat my assistant down the other day and helped her connect her workspace with Claude so she could work more efficiently. She had no idea she could do half the things I showed her. Nor did she think they were possible on the \$20/mo plan.

There's something almost *cosmically inevitable* about the rate of expansion of these tools. I've had countless conversations with folks (industry adjacent and otherwise) over the last few months where I've said: *We could stop at Fable and I could use this thing until the end of time, and never feel like I was hitting its limits*. The biggest hindrance now is speed — these things are slow. Check out [chatjimmy.ai](#) — a model ([burned into silicon](#)) that cranks out 17,000 tokens a second. Claude gives ya seventy a second if you're lucky. Yes, Jimmy is (kinda) dumb as rocks, but you can begin to imagine an avalanche of what-ifs — Fable at this speed would itself be an entirely new class of thinking tool.

What I most acutely lack when working on big LLM-built projects is a macro-scale overview. Here is my pie-in-the-sky setup:

- A giant wall, “blackboard” (E Ink?) preferably, that is a digital, infinitely zoomable canvas
- All the major components of the projects are visible (Mermaid-style, boxes-and-arrows diagrams), connections, dataflows, etc.
- You'd just stand in front of it and point and riff: What's happening over in this piece? Where's that data coming from? Which API?
- You could have the interface rendered as well, and talk through the design: Make these headers bigger, let's use sans serif fonts here, can we do a more playful animation moving between these sections?
- But most importantly, you could have collaborators stand there with you and talk through it all, pointing, riffing, all the while Claude or whatever listens, responds, implements

This kind of interface, combined with thousands or tens of thousands of tokens per second response times, is a tool I look forward to using.

Alright, that's all I got. Things are weird. Play with the weirdness. These tools are powerful and getting more powerful by the day. The valuations today seem irrational; the use-potential is not. Markets don't know how to price our (rapidly approaching) future.

Back to the writing caves,
C

[Craig Mod](#) is a writer and photographer based in Japan. He's the author of the books *Things Become Other Things* and *Kissa by Kissa* and is a MacDowell, Ragdale, and VCCA writing fellow. His essays and articles have appeared in *The New York Times*, *Eater*, *The Atlantic*, *California Sunday Magazine*, *Wired*, *Aeon*, *New Scientist*, *Virginia Quarterly Review*, *The New Yorker*, *The Morning News*, *Codex: Journal of Typography*, and elsewhere.

He writes newsletters, oh yes, [newsletters](#): [Roden & Ridgeline](#).

The work on this site is supported 100% by [paid memberships](#) and [book & print sales](#).

Search has its own bitter lesson

DOUG TURNBULL · 22 JUN 2026 · [SOURCE](#)

An agent with grep does quite well at search.

It's shocking to search technologists. We want to cocoon the problem in technology. We care about algorithms, knowledge graphs, ranking: ever-and-ever smarter retrieval.

It turns out, though, you can play a bit of a trick. If you convince everyone to optimize content for your search engine, you'll have built the best search engine.

There's Sutton's [bitter lesson](#) about unleashing raw compute on a problem. But there's a different bitter lesson in search: algorithms matter less than convincing the world to optimize for your search engine.

Sure, technology acts as the trigger. In Web Search, Google shifted away from text relevance to prioritize authoritativeness (PageRank). Their success told the market: create content others want to link to. They reshaped the Web for good (or ill) around that incentive.

Google crowdsourced search quality to the Web. Claude Code outsources it to every developer.